ConfigStream

Scalable Configuration Change Storage and Streaming Analytics

Phạm Công Hoàng Lê Tiến Hợp
Mai Vũ Hoàng Nguyễn Đức Dũng
Group 4 · Big Data Storage & Processing

First milestone checkpoint · 26 October 2026



Khoảng 25 giây. Chào thầy và các bạn. Nhóm 4 trình bày ConfigStream, hệ thống lưu trữ và phân tích thay đổi cấu hình. Buổi checkpoint này tập trung vào bài toán, dữ liệu, kiến trúc và những phần nhóm đã triển khai được. Chưa trình bày như một sản phẩm đã hoàn thiện hoặc đã chứng minh khả năng mở rộng.

The problem: configuration changes over time

Router configuration: version 10

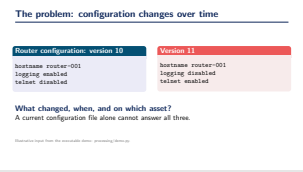
```
hostname router-001
logging enabled
telnet disabled
```

Version 11

```
hostname router-001
logging disabled
telnet enabled
```

What changed, when, and on which asset?
A current configuration file alone cannot answer all three.

Illustrative input from the executable demo: processing/demo.py.



Khoảng 45 giây. Mở đầu bằng ví dụ cụ thể: cùng một router nhưng phiên bản sau tắt logging và bật telnet. Đây là dữ liệu minh hoạ do nhóm tạo, không phải sự cố thực tế. Nếu chỉ giữ file hiện tại, ta không biết thay đổi xảy ra lúc nào và khác phiên bản trước ra sao. Nhóm muốn lưu lịch sử và chỉ ra phần thay đổi thay vì chỉ giữ trạng thái cuối cùng.

Four datasets, four meanings of change

Terraform / TerraDS: versions

Infrastructure code stored in Git.

v1 → **v2**: which lines changed?

RPKI: observed daily states

Observed route-authorization records.

Day 1 → **day 2**: which members differ?

RIS: routing events

BGP announcements and withdrawals.

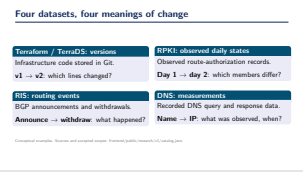
Announce → **withdraw**: what happened?

DNS: measurements

Recorded DNS query and response data.

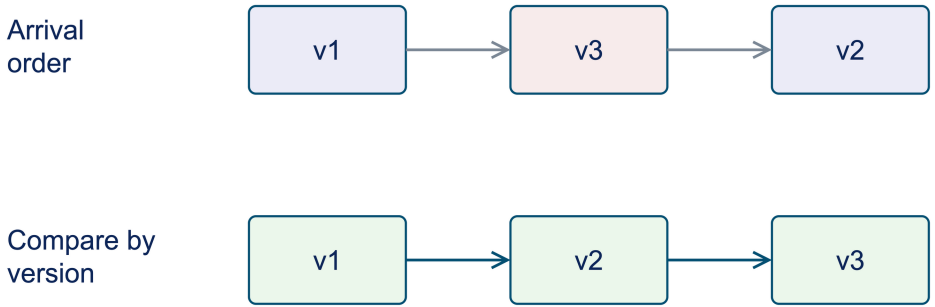
Name → **IP**: what was observed, when?

Conceptual examples. Sources and accepted scopes: frontend/public/research/v1/catalog.json.



Khoảng 65 giây. Nhóm dùng bốn loại dữ liệu có ý nghĩa thời gian khác nhau. Terraform từ TerraDS là các phiên bản file mã hạ tầng trong Git: so sánh để biết dòng nào đổi, không coi commit là lần triển khai. RPKI là tập bản ghi uỷ quyền định tuyến quan sát theo ngày; so sánh thành viên giữa hai ngày đầy đủ, không suy ra chính xác thao tác của quản trị viên. RIS ghi nhận sự kiện BGP, ví dụ quảng bá hoặc rút tuyến; hai sự kiện giống nội dung vẫn có thể là hai lần xảy ra khác nhau. DNS là phép đo truy vấn và phản hồi: thấy tên miền trả về địa chỉ nào tại thời điểm quan sát, không coi đó là toàn bộ trạng thái DNS. Ví dụ trên slide chỉ minh hoạ cách hiểu. Điểm chung là cần lưu lịch sử; cách định nghĩa thay đổi phải riêng cho từng loại.

Configuration example: handling late arrivals

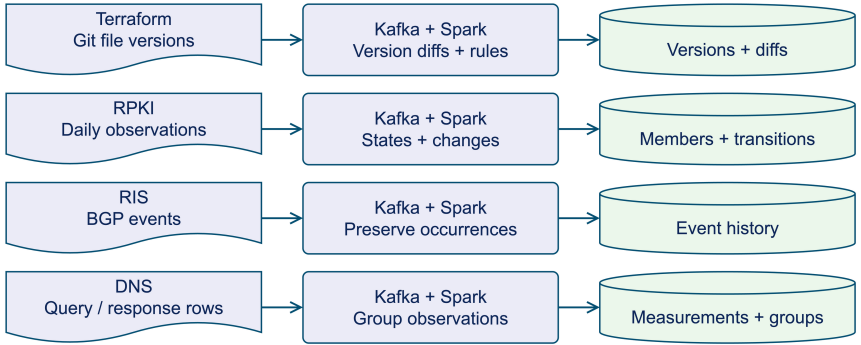


Compare **v1** → **v2** and **v2** → **v3**, once both endpoints are available.

processing/state.py; spark/streaming/stateful.py. Online repair is bounded by retained state.

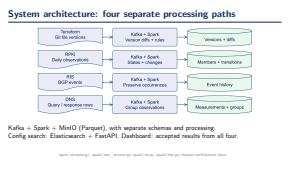
Khoảng 55 giây. Chỉ vào hàng trên: dữ liệu có thể đến theo thứ tự 1, 3, 2. Nếu so sánh theo thứ tự đến, ta sẽ tạo cặp 1 sang 3 và bỏ qua phiên bản 2. Hàng dưới là cách nhóm xử lý: giữ các phiên bản theo asset và chỉ tạo diff giữa các phiên bản liền kề. Nếu 2 đến khi state còn giữ được 1 và 3 thì tạo đủ hai cặp đúng. State có giới hạn lưu giữ, nên không hứa sửa được mọi dữ liệu đến muộn vô hạn.

System architecture: four separate processing paths



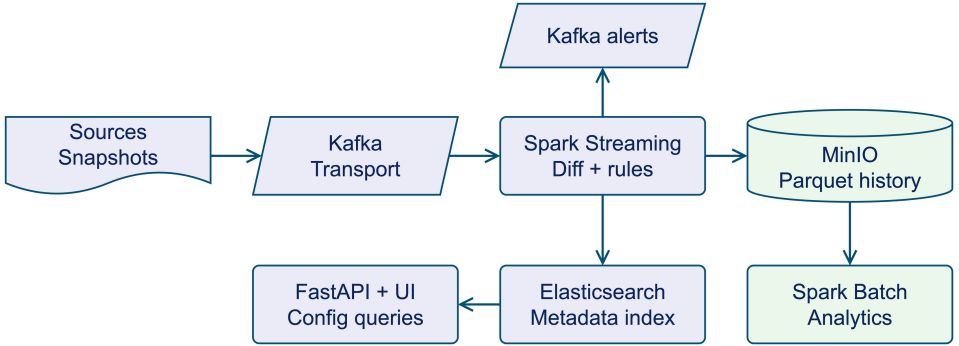
Kafka + Spark + MinIO (Parquet), with separate schemas and processing.
Config search: Elasticsearch + FastAPI. Dashboard: accepted results from all four.

spark/streaming/; spark/rpki_stream.py; spark/ris.py; spark/dns.py; domain architecture docs.



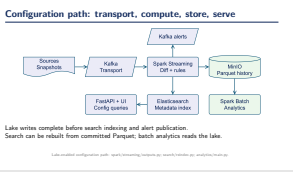
Khoảng 60 giây. Bốn hàng là bốn đường xử lý đã có trong repo, không phải tất cả đều đi qua bộ diff cấu hình. Hàng Terraform giữ logic phiên bản và rule. RPKI xử lý tập thành viên của các ngày đầy đủ và thay đổi giữa chúng. RIS giữ các lần xảy ra sự kiện, chỉ hoà giải bản gửi lại cùng danh tính nguồn; không xoá sự kiện chỉ vì nội dung giống nhau. DNS giữ phép đo và nhóm kết quả có thông tin nguồn để so sánh mô tả. Các luồng dùng chung công nghệ Kafka, Spark và lưu Parquet trên MinIO nhưng có schema, topic và metadata commit riêng. Elasticsearch và FastAPI phục vụ đường cấu hình; dashboard hiện có bản tổng hợp bằng chứng của cả bốn miền, không phải API live chung cho mọi miền. Dữ liệu thực ở đây được phát lại từ các nguồn đã cố định.

Configuration path: transport, compute, store, serve



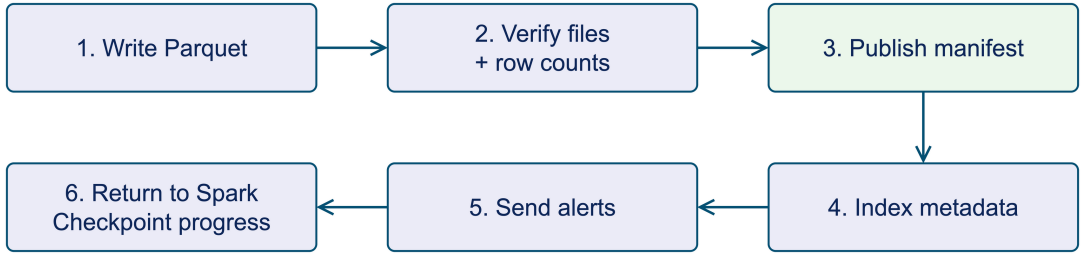
Lake writes complete before search indexing and alert publication.
Search can be rebuilt from committed Parquet; batch analytics reads the lake.

Lake-enabled configuration path: spark/streaming/outputs.py; search/reindex.py; analytics/main.py.



Khoảng 50 giây. Đi theo hàng chính từ trái sang phải: nguồn gửi snapshot qua Kafka, Spark xử lý state, diff và rule, rồi ghi vào MinIO. Nhánh dưới bên phải là Spark batch đọc lịch sử đã commit để phân tích. Nhánh search giữ metadata phục vụ FastAPI và giao diện, không chứa toàn bộ nội dung cấu hình. Nhánh trên là cảnh báo Kafka. Các mũi tên biểu diễn đường dữ liệu; thứ tự thực thi trong callback là lake trước, search sau, rồi mới phát alerts. Khi index mất hoặc cần tạo lại, có lệnh reindex đọc committed Parquet. Đây là cấu hình bật lake và search, không nói mọi chế độ chạy đều bắt buộc có tất cả thành phần.

A commit manifest marks readable history

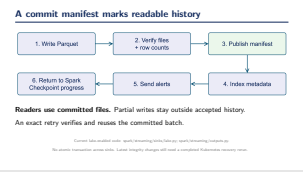


Readers use committed files. Partial writes stay outside accepted history.

An exact retry verifies and reuses the committed batch.

Current lake-enabled code: spark/streaming/sinks/lake.py; spark/streaming/outputs.py.

No atomic transaction across sinks. Latest integrity changes still need a completed Kubernetes recovery rerun.



Khoảng 45 giây. File xuất hiện trên MinIO chưa có nghĩa batch đã hoàn tất. Với code hiện tại, Spark ghi Parquet, lập inventory đường dẫn, kích thước và SHA-256, đọc lại theo schema để kiểm tra số dòng, rồi mới publish manifest. Reader chỉ đọc các đường dẫn được manifest khai báo. Sau đó callback mới index metadata, gửi alerts và trả quyền điều khiển để Spark tiến hành checkpoint. Nếu batch đã commit mà bước sau lỗi, retry kiểm tra đầy đủ projection, kể cả số lần xuất hiện của dòng, rồi tái sử dụng Parquet. Cảnh báo Kafka vẫn có thể lặp về mặt vật lý; không có giao dịch atomic xuyên các sink. Phần integrity mới đã có kiểm thử local nhưng lần recovery trên Kubernetes của image mới nhất chưa hoàn tất, nên không gán bằng chứng deployment cũ cho nó.

Prepared data: accepted scope for each dataset

Dataset	Accepted amount	Selected scope
Terraform	**10,188** Git versions	6,505 file histories / 375 repositories
RPKI	**14** complete daily observations	ripencc.tal, 1–14 Sep 2026
RIS	**4,725,156** event occurrences	rrc00, 1 Sep 2026, 00:00–01:00 UTC
DNS	**2,234,863** raw measurements	Two .li source objects, 1–2 Sep 2026

Controlled synthetic inputs also test retries and late arrivals.
Source manifests and result receipts are versioned in Git.

Accepted evidence: frontend/public/research/v1/catalog.json and its linked validation artifacts. Units differ; do not sum the rows.

Khoảng 55 giây. Đây là phạm vi đã được kiểm chứng trong các lần chạy ghi nhận, không phải tổng kích thước mọi nguồn ngoài Internet và không phải lần chạy mới hôm nay. Terraform có 10.188 phiên bản của 6.505 lịch sử file thuộc 375 repo. RPKI có 14 ngày đầy đủ trong phạm vi ripencc.tal; số 14 là số ngày quan sát, không phải số bản ghi bên trong. RIS có khoảng 4,73 triệu lần xảy ra sự kiện trong một giờ ở collector rrc00. DNS có khoảng 2,23 triệu dòng phép đo từ hai đối tượng nguồn .li đã chọn. Không cộng bốn con số vì đơn vị khác nhau. Nhóm còn có generator để chủ động kiểm thử dữ liệu lặp và đến muộn. Dữ liệu lớn lưu ngoài Git; repo giữ manifest và bằng chứng kết quả.

Implemented: differences and rule alerts

Diff: version 10 → 11

```
- logging enabled
- telnet disabled
+ logging disabled
+ telnet enabled
```

Rules detect

Logging disabled
High severity

Telnet enabled
Critical severity

Repeated delivery must not invent another logical change.

processing/demo.py; processing/diff.py; rules/default.yaml; tests/unit/test_rules.py.

Khoảng 45 giây. Quay lại ví dụ đầu: output chỉ ra các dòng bị bỏ và thêm. Rule mặc định báo logging bị tắt ở mức high và telnet được bật ở mức critical. Đây là rule minh hoạ đã có trong code, không phải mô hình AI dự đoán. Chuẩn hoá xử lý các khác biệt văn bản được hỗ trợ; không phải parser hiểu mọi ý nghĩa cấu hình. Việc gửi lại cùng snapshot cũng không được tạo thêm thay đổi logic mới.

Progress: runnable code and a recorded pipeline replay

Repository

`generator/`	input data
`processing/`	diff and rules
`spark/`	streaming jobs
`api/`	read API
`frontend/`	dashboard

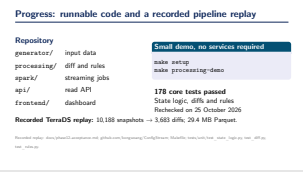
Small demo, no services required

```
make setup
make processing-demo
```

178 core tests passed
State logic, diffs and rules
Rechecked on 25 October 2026

Recorded TerraDS replay: 10,188 snapshots → 3,683 diffs; 29.4 MB Parquet.

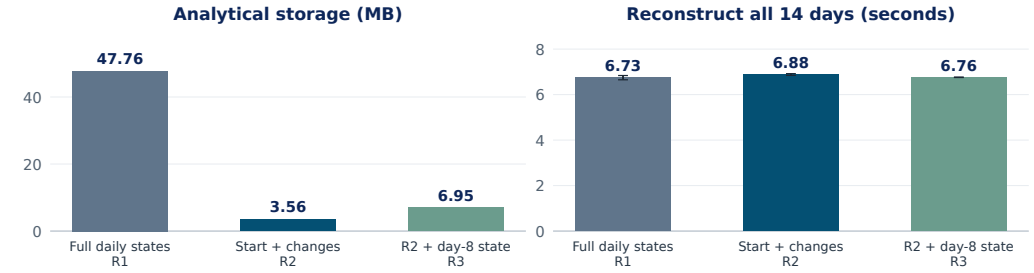
Recorded replay: docs/phase12-acceptance.md; github.com/kongwoang/ConfigStream; Makefile; tests/unit/test_state_logic.py, test_diff.py, test_rules.py.



Khoảng 60 giây. Repo có các thành phần ở cột trái. Hai lệnh bên phải chạy demo Python không cần service; 178 test của state, diff và rule đã được kiểm tra khi chuẩn bị slide. Dòng dưới là minh chứng tích hợp đã lưu: TerraDS đi qua Kafka, Spark và MinIO với 10.188 snapshot, 3.683 cặp liền kề và 29.402.997 byte Parquet. Phân biệt rõ demo nhỏ có thể chạy ngay với replay lịch sử đã được ghi nhận, không gọi đây là lần replay mới hôm nay. Nếu thầy yêu cầu demo trực tiếp, chuyển sang terminal đã chuẩn bị; không dùng kết quả này để suy ra HA hay scalability.

RPKI: less storage, similar reconstruction time

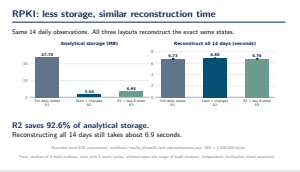
Same 14 daily observations. All three layouts reconstruct the exact same states.



R2 saves 92.6% of analytical storage.
Reconstructing all 14 days still takes about 6.9 seconds.

Recorded local E2R experiment: workloads/results/phase20/rpki-representations.json. MB = 1,000,000 bytes.

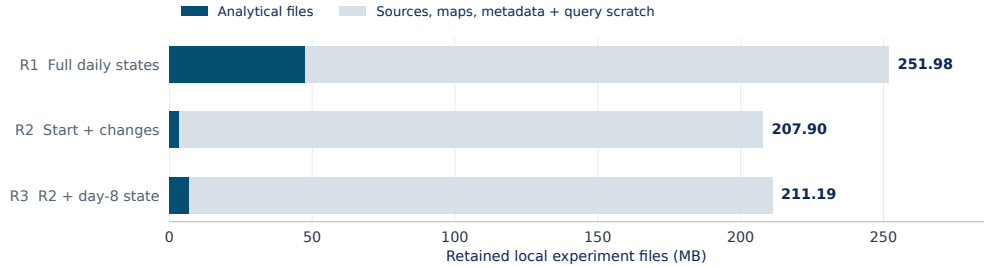
Time: median of 3 build medians, each with 3 warm cycles; whiskers show the range of build medians. Independent verification timed separately.



Khoảng 60 giây. Đây là phép so sánh trên cùng 14 ngày RPKI đã quan sát, không so số lượng bản ghi giữa bốn miền. R1 lưu toàn bộ tập thành viên của mỗi ngày. R2 lưu ngày đầu và các tập thêm, bớt giữa các ngày. R3 giống R2 nhưng thêm trạng thái đầy đủ ở ngày 8. Cả ba cùng giữ các transition sidecar chung, dùng cùng encoding và Snappy; đây là layout thí nghiệm local, chưa thay định dạng production. Biểu đồ trái: dung lượng phân tích giảm từ 47,76 MB xuống 3,56 MB ở R2, tức giảm 92,6 phần trăm. Biểu đồ phải: dựng lại đủ 14 ngày mất 6,73 giây ở R1 và 6,88 giây ở R2, gần tương đương trong phép đo này. R3 nằm giữa về dung lượng. Tất cả trạng thái và kết quả truy vấn khớp chính xác, không chỉ khớp số lượng. Kết luận: lưu thay đổi là ứng viên tốt cho lịch sử RPKI đã thử. Đây là ba build, mỗi build ba lượt truy vấn warm, không phải tốc độ Spark live hay phép đo cold cache.

The storage saving depends on what we count

Same RPKI experiment, now including the other retained files.

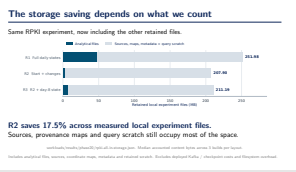


R2 saves 17.5% across measured local experiment files.

Sources, provenance maps and query scratch still occupy most of the space.

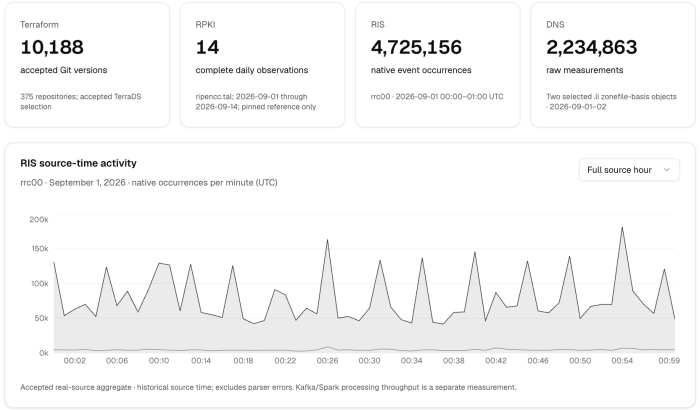
workloads/results/phase20/rpki-all-in-storage.json. Median accounted content bytes across 3 builds per layout.

Includes analytical files, sources, coordinate maps, metadata and retained scratch. Excludes deployed Kafka / checkpoint costs and filesystem overhead.



Khoảng 45 giây. Nếu chỉ nói tiết kiệm 92,6 phần trăm thì người nghe dễ hiểu thành toàn bộ hệ thống tiết kiệm như vậy. Khi cộng cả archive nguồn, ánh xạ về bản ghi gốc, metadata và file làm việc còn giữ lại, tổng file local giảm từ khoảng 251,98 MB xuống 207,90 MB, tức khoảng 17,5 phần trăm. Phần màu xanh đậm là dữ liệu phân tích đã tối ưu; phần xám vẫn lớn. Ý nghĩa thực tế: cần đo toàn bộ những thành phần đang giữ để quyết định tối ưu, không chỉ đo file đầu ra nhỏ nhất. Đây vẫn chỉ là tổng các thành phần của thí nghiệm biểu diễn local, chưa phải tổng disk của deployment vì không bao gồm Kafka, checkpoint hay overhead hệ thống file. Không suy kết quả RPKI thành mức tiết kiệm cho Terraform, RIS hoặc DNS.

Dashboard: four domains in one evidence overview

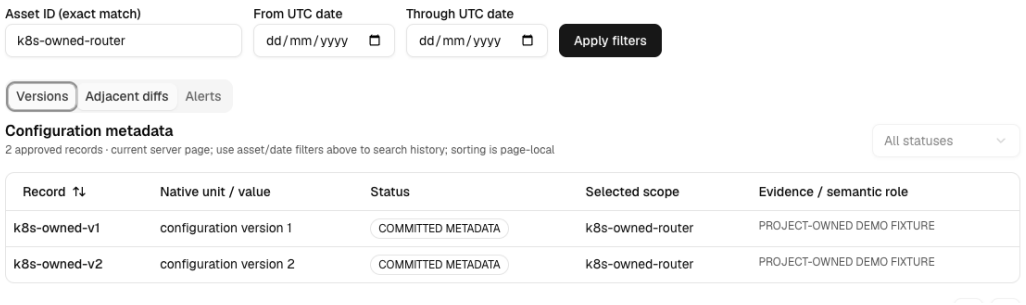


Actual frontend captured on 25 October 2026. This overview displays the accepted evidence bundle, not a live replay.



Khoảng 30 giây. Đây là ảnh giao diện thật được mở từ frontend hiện tại. Bốn thẻ phía trên tương ứng Terraform, RPKI, RIS và DNS vừa giới thiệu. Biểu đồ bên dưới là số sự kiện RIS theo thời gian nguồn, không phải throughput xử lý của Spark. Phần lịch sử cấu hình có API để tra cứu theo asset và thời gian khi backend chạy. Các con số trên ảnh thuộc những đơn vị khác nhau, không cộng lại thành một chỉ số tổng. Ảnh này không chứng minh backend đang chạy trực tiếp trong buổi checkpoint.

Dashboard: inspect configuration history by asset



1. **Filter** by asset and UTC dates. 2. **Select** versions, adjacent diffs or alerts.
3. **Inspect** safe metadata from the configuration API.

Recorded Phase 20V UI capture, 14 October 2026; docs/phase20v-demo-script.md.

The demonstrated asset is an owned fixture with 2 versions / 1 diff. Raw configuration bodies are not returned.



Khoảng 35 giây. Trang lịch sử cấu hình có ba thao tác chính: lọc asset và ngày UTC, chọn tab versions hoặc adjacent diffs hoặc alerts, rồi xem metadata trả về từ API. Ảnh chụp là buổi rehearsal đã ghi nhận ngày 14 tháng 10, với asset do nhóm tạo có hai phiên bản và một diff. Con số TerraDS ở thẻ tổng quan là aggregate đã lưu, không có nghĩa toàn bộ lịch sử TerraDS được phục vụ qua API này. Trang history cần backend đang chạy; ảnh không chứng minh API hiện tại đang online.

Checkpoint readiness and next milestone

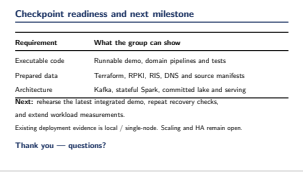
Requirement	What the group can show
Executable code	Runnable demo, domain pipelines and tests
Prepared data	Terraform, RPKI, RIS, DNS and source manifests
Architecture	Kafka, stateful Spark, committed lake and serving

Next: rehearse the latest integrated demo, repeat recovery checks,

and extend workload measurements.

Existing deployment evidence is local / single-node. Scaling and HA remain open.

Thank you — questions?



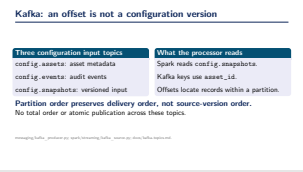
Khoảng 55 giây. Chốt lại ba yêu cầu của thầy: code chạy được, dữ liệu đã chuẩn bị, và kiến trúc đã có implementation cùng kết quả tích hợp. Nhóm đã chạy pipeline trước đây; bước tiếp theo là diễn tập lại trên image mới nhất, kiểm tra recovery và mở rộng phép đo. Không nói cả hệ thống production đã hoàn thành. Nếu thầy hỏi sâu về Kafka, state, layout Parquet hoặc dashboard experiments và operations thì chuyển sang các slide phụ phía sau.

Kafka: an offset is not a configuration version

Three configuration input topics	What the processor reads
`config.assets`: asset metadata	Spark reads `config.snapshots`.
`config.events`: audit events	Kafka keys use `asset_id`.
`config.snapshots`: versioned input	Offsets locate records within a partition.

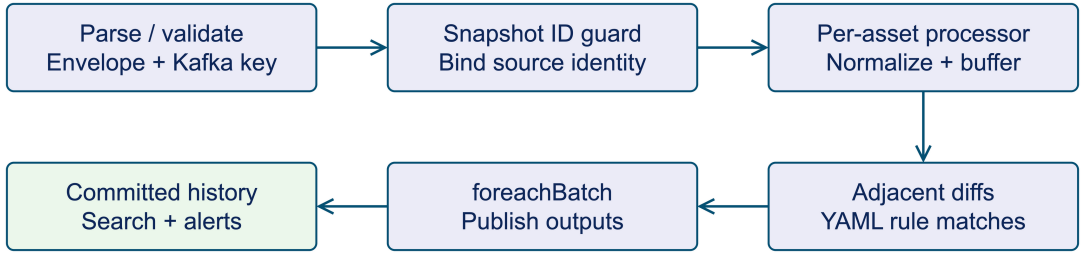
Partition order preserves delivery order, not source-version order.
No total order or atomic publication across these topics.

messaging/kafka_producer.py; spark/streaming/kafka_source.py; docs/kafka-topics.md.



Slide phụ, không tính vào thời gian trình bày chính. Kafka tách nguồn phát khỏi processor và cho phép replay khi record còn retention. Asset, event và snapshot là ba loại thông điệp riêng. Spark dùng snapshot để xử lý state; không đợi event ở topic khác xuất hiện trước. Cùng asset ID giúp phân vùng, nhưng producer có thể gửi v3 trước v2 nên offset không thay được số phiên bản. Không giả định thứ tự hoặc atomicity giữa các topic.

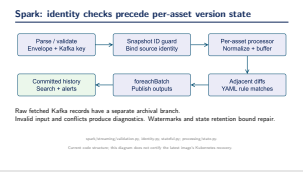
Spark: identity checks precede per-asset version state



Raw fetched Kafka records have a separate archival branch.
Invalid input and conflicts produce diagnostics. Watermarks and state retention bound repair.

spark/streaming/validation.py, identity.py, stateful.py; processing/state.py.

Current code structure; this diagram does not certify the latest image's Kubernetes recovery.



Slide phụ. Bước parse kiểm tra envelope, thời gian, hình thức hash và Kafka key. Sau đó IdentityProcessor group theo snapshot ID và kiểm tra binding về nguồn, gồm kiểm tra content hash qua schema; exact retry trong state được bỏ qua, conflicting identity thành diagnostic. Chỉ sau bước này mới group theo asset. Bên trong processor theo asset, code chuẩn hoá snapshot trước khi thêm vào buffer, rồi tạo các diff liền kề và áp dụng rule. Kiểm tra danh tính snapshot và sắp thứ tự phiên bản theo asset là hai bước riêng. Raw được lấy từ nhánh Kafka trước validation để còn giữ bằng chứng của bản ghi không được business logic chấp nhận.

Parquet layout separates raw evidence and processed history

```
config-lake/
  raw/snapshots/
  processed/snapshots/
  processed/diffs/
  processed/alerts/
  processed/diagnostics/
  _commits/
  analytics/
```

Raw: binary Kafka key/value, topic, partition, offset and timestamp.

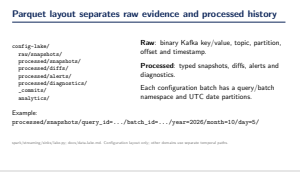
Processed: typed snapshots, diffs, alerts and diagnostics.

Each configuration batch has a query/batch namespace and UTC date partitions.

Example:

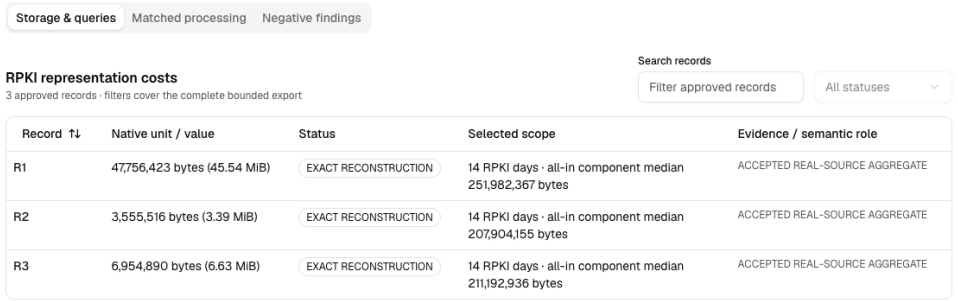
`processed/snapshots/query_id=.../batch_id=.../year=2026/month=10/day=5/`

spark/streaming/sinks/lake.py; docs/data-lake.md. Configuration layout only; other domains use separate temporal paths.



Slide phụ. Đây là layout của đường cấu hình, không phải schema chung cho cả bốn loại dữ liệu. Raw giữ byte Kafka và provenance; processed giữ các output đã chuẩn hoá. Query ID và batch ID xác định vùng ghi, còn year/month/day giúp đọc theo ngày. Ngày của raw lấy từ Kafka timestamp; ngày của các output khác lấy theo loại record. Kafka partition là phân vùng log, Spark execution partition là phân công xử lý, date partition là tổ chức file: ba khái niệm khác nhau. Chỉ raw được query fetch trong retention mới có thể lưu; không thể khôi phục record đã bị Kafka xoá trước khi đọc.

Dashboard: inspect the evidence behind a comparison



Storage & queries | Matched processing | Negative findings

Search records

Filter approved records | All statuses

RPKI representation costs

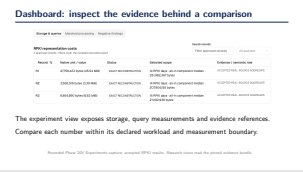
3 approved records · filters cover the complete bounded export

Record ↑↓	Native unit / value	Status	Selected scope	Evidence / semantic role
R1	47,756,423 bytes (45.54 MiB)	EXACT RECONSTRUCTION	14 RPKI days · all-in component median 251,982,367 bytes	ACCEPTED REAL-SOURCE AGGREGATE
R2	3,555,516 bytes (3.39 MiB)	EXACT RECONSTRUCTION	14 RPKI days · all-in component median 207,904,155 bytes	ACCEPTED REAL-SOURCE AGGREGATE
R3	6,954,890 bytes (6.63 MiB)	EXACT RECONSTRUCTION	14 RPKI days · all-in component median 211,192,936 bytes	ACCEPTED REAL-SOURCE AGGREGATE

The experiment view exposes storage, query measurements and evidence references.

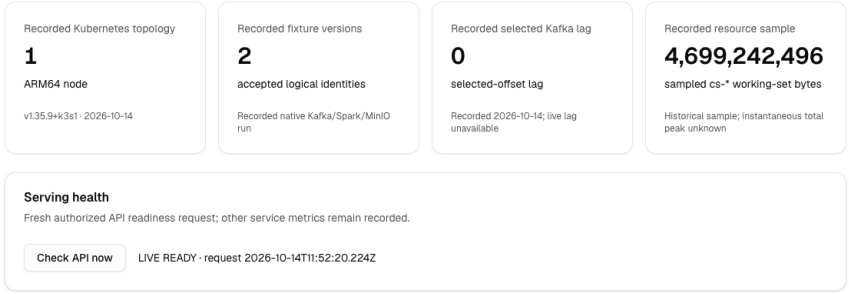
Compare each number within its declared workload and measurement boundary.

Recorded Phase 20V Experiments capture; accepted RPKI results. Research views read the pinned evidence bundle.



Slide phụ. Đây là màn hình experiments để đối chiếu kết quả với phạm vi và bằng chứng của phép đo. Bảng hiển thị R1, R2, R3 với byte phân tích và tổng component đã đo, có scope và bằng chứng. Các tab khác cho phép xem matched processing và negative findings. Đây là dữ liệu thí nghiệm đã được export và kiểm tra hash, không phải một job benchmark vừa chạy khi mở trang. Kết quả RPKI khớp hai biểu đồ trong phần trình bày chính.

Dashboard: distinguish recorded runs from an API probe



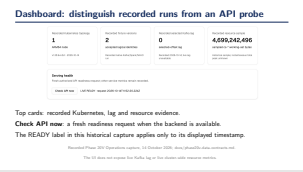
Top cards: recorded Kubernetes, lag and resource evidence.

Check API now: a fresh readiness request when the backend is available.

The READY label in this historical capture applies only to its displayed timestamp.

Recorded Phase 20V Operations capture, 14 October 2026; docs/phase20v-data-contracts.md.

The UI does not expose live Kafka lag or live cluster-wide resource metrics.



Slide phụ. Ảnh có chữ LIVE READY nhưng đó là trạng thái của request tại timestamp trong ảnh, không phải trạng thái hiện tại. Các thẻ node, Kafka lag và resource là bằng chứng chạy đã lưu. Khi demo thật, bấm Check API now mới tạo request readiness và giao diện làm hết hiệu lực kết quả sau thời gian quy định. Readiness thành công chứng minh API và dependencies cần thiết cho request đó, không chứng minh toàn bộ pipeline đang ingest dữ liệu. Dashboard không có quyền đọc kubeconfig hay Kafka credentials và không phải một hệ thống monitoring cluster trực tiếp.